

The *Self-Attention* Layer in Transformer Models

A rigorous exploration of the Query-Key-Value mechanism, scaled dot-product attention, multi-head formulations, and computational properties that underpin modern large language models.

- Uditya Narayan Tiwari
- June 2026
- Transformer Architecture
- Deep Learning

CONTENTS

- | | | | |
|----|-------------------------------|----|-----------------------|
| 01 | Motivation & Intuition | 02 | The QKV Framework |
| 03 | Scaled Dot-Product Attention | 04 | Multi-Head Attention |
| 05 | Attention Score Visualisation | 06 | Positional Encodings |
| 07 | Computational Complexity | 08 | Variants & Extensions |

01

Motivation & Intuition

Why attention? What does it solve?

Prior to Transformers, sequence models (RNNs, LSTMs) processed tokens **sequentially**, forcing information to travel through a chain of hidden states. This creates two critical problems: **vanishing gradients** over long sequences, and an inability to parallelise computation during training.

The self-attention mechanism replaces sequential processing with a fully-connected, parallelisable computation over the entire sequence. Every token in the sequence directly attends to every other token in **$O(1)$ distance**, irrespective of how far apart they are in the sequence.

CORE INTUITION

"How relevant is each other word in this sentence to understanding the current word?" — Self-attention answers this question for every token simultaneously, producing a context-aware representation for each position.

Consider the sentence *"The animal didn't cross the street because it was too tired."* To resolve what "it" refers to, a model must link "it" back to "animal" — a long-range dependency. Self-attention assigns high weight to that connection directly.

02

The Query-Key-Value Framework

Projecting representations into three distinct roles

Every input token embedding **x** is linearly projected into three distinct vector spaces using learned weight matrices — producing Queries (Q), Keys (K), and Values (V). This is the foundational trick.

```

Q = X · WQ    // "What am I looking for?"
K = X · WK    // "What do I contain?"
V = X · WV    // "What do I share if selected?"

```

where $W_Q, W_K, W_V \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$

Q

Query Vector

Represents the current token's "information request". It asks: which other positions in the sequence are relevant to me? Queries interact with all Keys to compute raw attention scores.

K

Key Vector

Acts as an index or label for each token. It answers: what kind of information do I hold? The dot product of a Query with all Keys reveals compatibility scores (affinities).

V

Value Vector

Contains the actual information to be aggregated. Once attention weights are computed (from Q:K), they are applied to Values via a weighted sum to produce the final contextualised output.

03

Scaled Dot-Product Attention

The core mathematical operation

Given the Q, K, V matrices for an entire sequence of n tokens, the attention output is computed in three steps: compute raw scores via dot products, scale and normalise with

softmax, and perform a weighted sum of Values.

CORE FORMULA

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

Dot-Product Scores: QK^T

①

For each token's Query, compute its dot product with every token's Key. This produces an $n \times n$ raw score matrix — higher scores indicate higher relevance between two positions. Complexity: $O(n^2 d_k)$.

Scaling by $\sqrt{d_k}$

②

Without scaling, dot products can grow large when d_k is high, pushing softmax into regions with extremely small gradients. Dividing by $\sqrt{d_k}$ keeps the variance of the scores ≈ 1 , stabilising training.

Softmax Normalisation

③

Softmax is applied row-wise to the score matrix, converting raw scores into a probability distribution. Each row sums to 1, representing how much attention token i pays to every other token j .

Weighted Sum of Values

④

The softmax weights are multiplied with the Value matrix V , producing a weighted mixture of all Value vectors. This is the contextualised output for each position — an information-rich blend of all relevant tokens.

WHY SCALING MATTERS

If $d_k = 64$, then $\sqrt{64} = 8$. Without scaling, QK^T values could be $\sim 64\times$ larger than intended, causing softmax to produce near-one-hot distributions. Scaling prevents this collapse, encouraging diffuse, expressive attention patterns.

In **decoder self-attention**, a causal mask is added before softmax — setting future positions to $-\infty$, ensuring tokens can only attend to past positions. This preserves the autoregressive property during generation.

MASKED VARIANT

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right) \cdot V$$

where $M[i, j] = -\infty$ if $j > i$, else 0

04

Multi-Head Attention

Attending across multiple representation subspaces

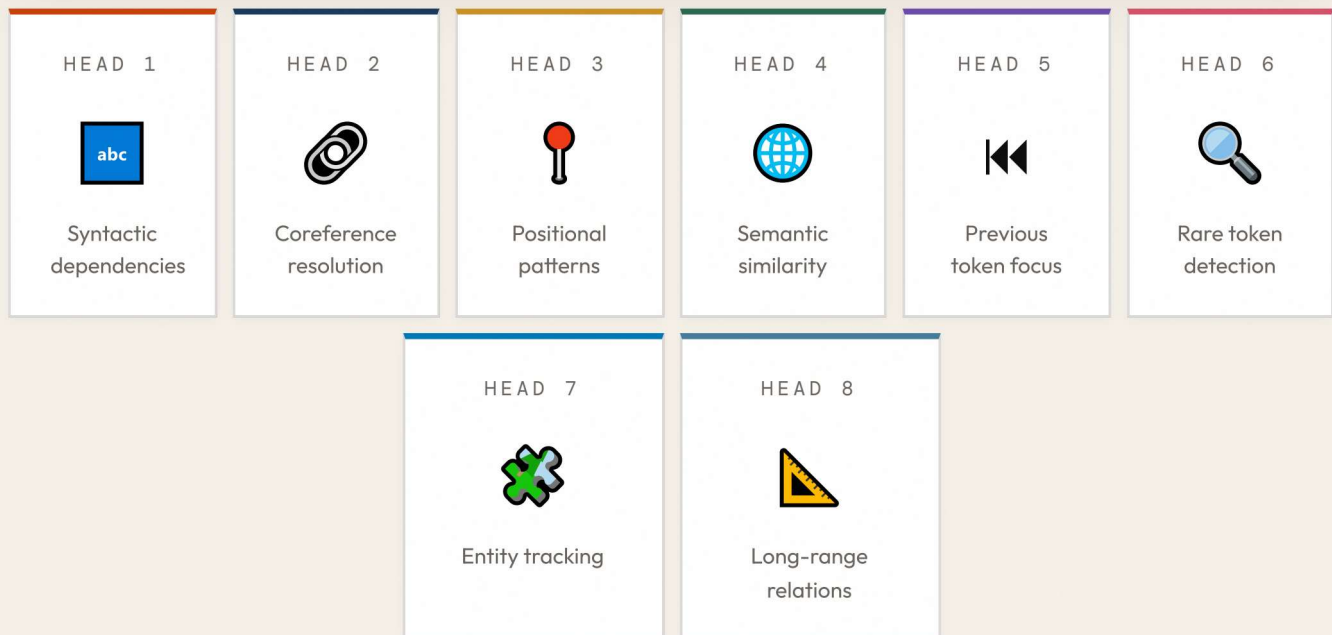
A single attention head computes one set of attention weights — one "view" of token relationships. In practice, different aspects of linguistic structure (syntax, coreference, semantics) benefit from different patterns of attention simultaneously. Multi-head attention achieves this.

The full model dimension d_{model} is split into h heads. Each head operates with $d_k = d_{\text{model}} / h$ dimensions, meaning total parameters remain constant. The h outputs are concatenated and linearly projected.

MULTI-HEAD FORMULA

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_0$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



Concatenate all heads $\rightarrow$ Project via $W_O \in \mathbb{R}^{(h \cdot d_v \times d_{\text{model}})}$

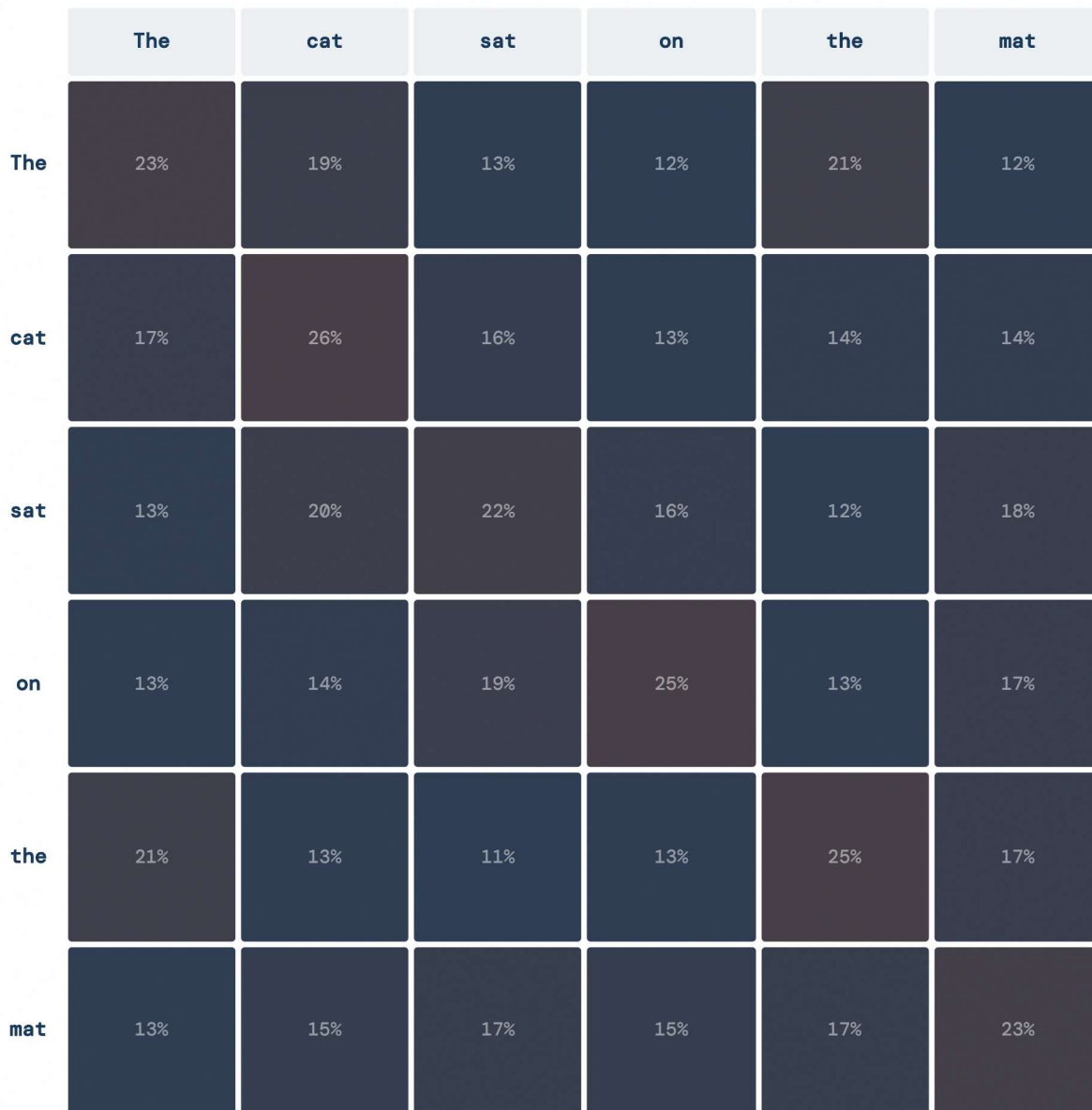
The original "Attention is All You Need" paper (Vaswani et al., 2017) used **$h = 8$ heads**, $d_{\text{model}} = 512$, so each head had $d_k = d_v = 64$. Modern large models like GPT-4 and Claude use significantly more heads and larger model dimensions.

05 Attention Score Visualisation

An interactive heatmap of simulated attention weights

The softmax output produces an $n \times n$ matrix of attention weights. Each row is a probability distribution over all tokens. The heatmap below simulates what an attention head might look like for a 6-token sequence. Hover over cells to inspect individual scores.

Simulated Attention Weights — Single Head



Tokens: [The] [cat] [sat] [on] [the] [mat] · Darker = stronger attention ·
Rows = Query, Cols = Key

Research tools like BertViz and attention-analysis packages allow practitioners to visualise these matrices in trained models, revealing interpretable patterns such as heads that track subject-verb agreement or attend to the previous token.

06

Positional Encodings

Injecting sequence order into a permutation-invariant operation

Self-attention is inherently **permutation-invariant** — if you shuffle the input tokens, the attention computation produces the same results (just in a different order). The operation has no built-in sense of which token comes first. This is a problem for language, where order is critical.

Positional encodings add position information to the token embeddings *before* the attention layer. The original paper used sinusoidal encodings:

SINUSOIDAL POSITIONAL ENCODING

```
PE(pos, 2i) = sin(pos / 100002i/d_model)
PE(pos, 2i+1) = cos(pos / 100002i/d_model)

pos = token position, i = dimension index
```

METHOD	DESCRIPTION	GENERALISES TO UNSEEN LENGTHS?
Sinusoidal (abs.)	Fixed sin/cos patterns; no learned parameters	✓ Yes (by formula)
Learned (abs.)	Trainable embedding per position (GPT-2 style)	✗ No (bounded by training length)
RoPE	Rotary PE: encodes relative positions via rotation matrices	✓ Strong generalisation

METHOD	DESCRIPTION	GENERALISES TO UNSEEN LENGTHS?
ALiBi	Attention with Linear Biases; no encoding vectors needed	✓ Excellent length extrapolation
YaRN / LongRoPE	Extended RoPE scaling for context lengths of 128K+	✓ Designed for ultra-long context

07

Computational Complexity

The quadratic bottleneck and its implications

The self-attention mechanism has $O(n^2 \cdot d)$ time and memory complexity — quadratic in sequence length n . This arises because every token attends to every other token: n tokens $\times$ n tokens $\times$ d -dimensional products.

LAYER TYPE	TIME COMPLEXITY	MEMORY
Self-Attention	$O(n^2 \cdot d)$	$O(n^2)$
RNN / LSTM	$O(n \cdot d^2)$	$O(1)^*$
Conv Layer	$O(k \cdot n \cdot d^2)$	$O(k)$
Sparse Attn.	$O(n \sqrt{n} \cdot d)$	$O(n \sqrt{n})$
Flash Attn.	$O(n^2 \cdot d)$	$O(n)$ ✓

The quadratic scaling is the primary reason why processing very long sequences (e.g., 100K tokens) is expensive. **FlashAttention** (Dao et al., 2022) keeps the same $O(n^2 \cdot d)$ compute but

reduces HBM memory access to $O(n)$, making long contexts practical in GPU memory.

FLASHATTENTION

SPARSE ATTENTION

LINEAR ATTENTION

RING ATTENTION

 are all research directions aimed at breaking the quadratic barrier, each with different trade-offs in approximation quality, hardware efficiency, and implementation complexity.

08

Variants & Extensions

How self-attention has evolved across architectures

Since the original Transformer, numerous architectural variants have modified or extended the self-attention mechanism to improve efficiency, expressiveness, or specialised capabilities.

VARIANT	KEY CHANGE	USED IN
Cross-Attention	Q from decoder, K/V from encoder — bridges two sequences	Seq2seq, T5, BART
Grouped-Query Attn (GQA)	Multiple Q heads share one K/V pair — reduces KV cache size	Llama 2/3, Mistral
Multi-Query Attn (MQA)	All Q heads share a single K/V — maximum cache efficiency	PaLM, Falcon
Sliding Window Attn	Each token attends only to a local window of w neighbours	Mistral, Longformer
Linear Attention	Kernel trick replaces softmax — reduces to $O(nd^2)$	RWKV, RetNet

VARIANT	KEY CHANGE	USED IN
Differential Attention	Two softmax maps subtracted — reduces attention noise	Microsoft Research 2024

SUMMARY

Self-attention is not a single fixed mechanism — it is a design space. The core QKV intuition remains universal, but every modern architecture innovates on the how: how keys and values are shared, how positions are encoded, how the attention pattern is constrained, and how memory is managed during inference.

Uditya Narayan Tiwari

SOFTWARE DEVELOPER · AI / ML ENTHUSIAST

PORTFOLIO https://lnkd.in/dae3_HZy

GITHUB <https://lnkd.in/eJBGnzbc>

LINKEDIN </in/uditya-narayan-tiwari-562332289>

RESOURCES udityaknowledgebase.netlify.app

JUNE 2026

*Self-Attention in
Transformer Models*

SELF-ATTENTION